

AI-OCR Receipt Extraction Website

Carbon Crunch Shortlisting Assignment | Short Technical Documentation

i. Approach

The system implements a five-stage modular pipeline that converts raw receipt images into structured, confidence-annotated JSON outputs and aggregated financial summaries.

Stage	Module	Description
1. Preprocessing	preprocessor.py	Grayscale, denoise (NL-means), deskew (Hough), CLAHE contrast, adaptive binarization
2. OCR	ocr_engine.py	EasyOCR (primary) or Tesseract (fallback) — returns per-word confidence scores
3. Extraction	extractor.py	Regex + keyword heuristics extract store name, date, line items, and total amount
4. Structuring	extractor.py	Typed dataclasses produce JSON with per-field confidence + low-confidence flags
5. Summarization	summarizer.py	Aggregates totals, per-store spend, avg/min/max, and flags unresolved receipts

Confidence Scoring Model

Each extracted field carries a composite confidence score combining three signals:

Signal	How it is used
OCR confidence	Raw per-token probability from EasyOCR / Tesseract (0–1)
Pattern validation	Regex match strength for dates (5 formats) and currency (4 formats)
Keyword heuristics	Proximity of anchors like 'Total', 'Amount Due', 'Grand Total'

Fields with confidence < 0.70 are flagged in `flagged_fields[]`. The `overall_confidence` is the mean of all resolved field scores

ii. Tools Used

Tool / Library	Version	Role
EasyOCR	≥ 1.7.0	Primary OCR engine — handles diverse fonts and skewed text with native confidence
pytesseract	≥ 0.3.10	Fallback OCR engine — word-level confidence via <code>image_to_data()</code>
OpenCV (cv2)	≥ 4.8.0	Image preprocessing: denoising, Hough-based deskew, CLAHE, adaptive threshold
Pillow	≥ 10.0.0	Image loading and format conversion
NumPy	≥ 1.24.0	Array operations throughout preprocessing and OCR grouping
ReportLab	≥ 4.0	PDF generation (this document)
Python stdlib	3.9+	<code>re</code> , <code>json</code> , <code>pathlib</code> , <code>argparse</code> , <code>dataclasses</code> , <code>logging</code> , <code>collections</code>

iii. Challenges Faced

- **Skew detection noise** — Receipts with dominant vertical lines (price columns, borders) produced false skew angles. Resolved by filtering Hough lines to $\pm 45^\circ$ and applying a 0.5° minimum threshold before rotation.
- **Store name ambiguity** — Receipt headers mix store names with phone numbers, URLs, and promotional slogans. Resolved using a multi-pattern exclusion regex (phone, URL, tax keywords) combined with a line-length filter and ALL-CAPS / Title Case preference.
- **Currency format variation** — Real-world receipts use \$, £, €, Rs., INR, and mixed comma/dot separators. Resolved with a layered regex parser that strips non-numeric characters and handles thousand separators before float conversion.
- **OCR word grouping** — EasyOCR returns phrase-level bounding boxes, not lines, making reconstruction of item-price pairs unreliable. Resolved by clustering detections on Y-coordinate proximity (12 px tolerance) and re-sorting by X within each line.
- **Low-quality / partial images** — Blurred, folded, or partially cropped receipts produce sparse OCR output. Resolved by graceful degradation: each field returns None with confidence 0.0 rather than crashing, and such receipts are listed in `unresolved_receipts[]`.

iv. Improvements

- **Fine-tuned document model** — Replace regex heuristics with a fine-tuned LayoutLM or Donut model trained on CORD / SROIE receipt datasets for layout-aware, end-to-end extraction.
- **Confidence calibration** — Train a logistic regression calibrator on annotated ground-truth to map raw OCR scores to true accuracy probabilities, improving flag thresholds.
- **Multi-language support** — Extend EasyOCR language packs for Hindi, Arabic, and Chinese to handle multilingual receipts common in international deployments.
- **Currency normalisation** — Add automatic currency detection and conversion to a base currency (e.g., USD) using a live FX API, enabling accurate cross-receipt comparisons.
- **Human-in-the-loop queue** — Route receipts with `overall_confidence < 0.65` to a review UI where corrections are fed back as training signal, creating a continuous improvement loop.
- **Batch GPU inference** — Enable EasyOCR GPU mode and batch image processing to reduce per-receipt latency from ~ 2 s to < 0.3 s at scale.